

IMAGE PROCESSOR:
GRAYSCALE CONVERSION & HISTOGRAM EQUALIZATION
Sequential vs Parallel

1. Problem Statement & Motivation

The operations included in image processing such as grayscale conversion and histogram equalization are pixel-independent in nature, i.e., the output value at each pixel can be determined without the knowledge of any other pixel's result. Therefore, an image processor is an apt domain to demonstrate parallelism.

Grayscale Conversion and Histogram Equalization are two operations that are widely used in image editing tools, like Photoshop, medical imaging, surveillance and security footages, etc.

The parallel version of this image processor addresses two problems over the sequential version. Firstly, since the architecture of this image processor almost compatible for parallel work, a pure sequential version takes tens of milliseconds per stage to do the parallel pixel-work. Secondly, in entire histogram equalization specifically, the cumulative distributive function (CDF) had genuine data dependency between adjacent elements. Also, one major problem we faced was the occurrence of a race condition during histogram computation which solved later using mutex locks. Understanding and demonstrating both of these problems — what parallelizes cleanly and what does not, and why — was the core motivation for this project, more so than raw performance numbers in isolation.

1. Objectives

- Implement a correct, sequential baseline image processing pipeline consisting of grayscale conversion and histogram-equalization contrast enhancement, decomposed into individually measurable stages.
- Implement a parallel version of the same pipeline using explicit multithreading, preserving bit-for-bit identical output to the sequential baseline.
- Identify, from first principles rather than by default, which pipeline stages are embarrassingly parallel and which contain genuine data dependencies that make them unsuitable for parallelization.
- Design and justify a synchronization strategy for the one stage that requires shared, mutable state (histogram binning) that avoids serializing the parallel work it protects.
- Empirically measure per-stage execution time for both implementations across a range of image resolutions (8.3–24 megapixels) and thread counts, and use those measurements to explain — not just report — the achieved speedup.
- Frame all major design decisions (threading model, synchronization primitive, build system, I/O strategy) as justified trade-offs against explicitly considered alternatives, rather than as defaults.

1. Methodology and Design

This section documents why each major design decision was made over the alternatives that were considered. Per-decision trade-off reasoning is the most heavily weighted part of this report, so each subsection below states the alternative(s) considered, why they were rejected, and what evidence (empirical or structural) supports the choice that was made.

1. Pipeline Architecture

Our design is structured as a seven stage linear pipeline: image loading, grayscale conversion, histogram computation, CDF computation, LUT construction, LUT application and image saving. By choosing this design, we were able to structurally arrange the parts of each implementation, which thereby, gave a better understanding of how the program works.

The decomposition of histogram equalization was done into its four constituent sub-steps(histogram computation, CDF computation, LUT construction, LUT application) rather than one opaque `equalizeHistogram()` because each step's parallelizability was independent. Treating equalization as a black box would have hidden the fact that it contains both a fully parallel stage and a strictly sequential one, which was one of the central findings of this project.

1. Threading Model: Pthreads over OpenMP

Both an OpenMP version and a pthreads version were implemented and benchmarked before settling on pthreads for the submitted implementation. This was a genuine comparison, not an assumption, and it is worth stating both sides plainly:

- **OpenMP** expresses parallelism declaratively via compiler pragmas (`#pragma omp parallel for`), letting the compiler and runtime handle thread creation, work distribution, and variable privatization automatically. This made the grayscale and LUT-application stages nearly identical to their sequential counterparts, with a single line of annotation — objectively less code and less opportunity for concurrency bugs.
- **Pthreads** requires explicitly creating each thread (`pthread_create`), manually partitioning the workload across them, and joining them (`pthread_join`) before using their results. This is more verbose and carries more risk of concurrency bugs, but it makes the thread lifecycle and synchronization strategy an explicit, inspectable part of the code rather than a compiler detail.

Pthreads was chosen for the final implementation for two concrete reasons:

- **Learning objective and inspectability.** As a team new to parallel programming, we deliberately chose the more manual approach because it forces every synchronization decision to be visible in our own code rather than delegated to a compiler pragma. `#pragma omp critical` solves the histogram race condition just as correctly as a pthreads mutex, but it does so opaquely. Implementing the lock explicitly with `pthread_mutex_t`, `pthread_mutex_lock`, and `pthread_mutex_unlock` makes the race condition it prevents — and the reasoning behind the fix — something we had to understand and design ourselves, which was judged more valuable for this assignment's learning goals than OpenMP's convenience.
- **Finer control over thread count and row partitioning,** which benchmarking depends on directly. The same binary accepts an explicit thread-count argument (`argv[3]`) without needing environment variables (`OMP_NUM_THREADS`) or runtime-specific APIs, which simplified the benchmarking harness used to produce the results in Section 6.

The accepted design is a larger surface area for concurrency bugs- a manually managed mutex can be forgotten or misplaced in a way a `#pragma` cannot. This was mitigated by verifying pixel-identical output against the sequential baseline for every test image (Section 6), and was judged an acceptable cost given the explicit control was more valuable for this project's learning objectives than OpenMP's convenience.

1. Mutex-Protected Reduction over Atomics

The histogram computation stage is the only place in the pipeline where multiple threads write to shared state, and it was the one place where a synchronization primitive had to be chosen deliberately. Three approaches were considered:

- **Per-pixel mutex lock — rejected outright.** Calling `pthread_mutex_lock()` before every single `histogram[pixelValue]++` would be correct but would serialize the entire pixel scan behind one lock, since every thread would contend for it on every pixel. For a 24-megapixel image this is 24 million lock/unlock pairs — the synchronization overhead would dominate and the parallel version could plausibly run slower than the sequential one.
- **Atomic increment per pixel (`std::atomic` on shared bins) — implemented, benchmarked informally, and rejected.** During development this version measurably increased processing time for the histogram stage relative to the reduction approach described below — the fine-grained atomic accesses turned what should be a fully parallel, lock-free scan into a stage bottlenecked by cache-coherency traffic between cores, once per pixel rather than once per thread.
- **Thread-local accumulation with a single coarse-grained merge — the approach used in the submitted code.** Each thread accumulates into a private, thread-local 256-bin histogram while scanning its assigned rows (no locking required, since no other thread can see this memory), and only locks the shared mutex once, at the end, to merge its 256 local counts into the shared result.

The reasoning for preferring the reduction pattern over atomics is structural, independent of the specific numbers observed: it confines synchronization to a fixed, small cost — 256 additions under one lock, per thread, performed once — regardless of image size, while the expensive part (scanning up to 24 million pixels) runs fully parallel with zero lock contention and zero shared-memory writes during the scan itself. An atomic-per-pixel approach pays a synchronization cost that scales with the number of pixels; the reduction pattern pays a synchronization cost that scales only with (thread count × 256 bins), which is asymptotically far cheaper and matches what was observed empirically during development.

1. An Amdahl's Law-Driven Design

Not every stage was parallelized, and this was a conscious design decision rather than an oversight. CDF computation and LUT construction were deliberately left sequential:

- CDF computation has an inherent data dependency — `cdf[i]` requires `cdf[i-1]` — making it a genuine prefix-sum problem. Parallel prefix-sum algorithms exist (e.g. Hillis-Steele, Blelloch), but they only pay off at scale.
- Both CDF and LUT construction operate on a fixed 256 elements regardless of image size (one bin per 8-bit intensity value). Thread-creation overhead for even two threads would exceed the total work being parallelized.

This is directly measurable rather than assumed: the instrumented per-stage breakdown (Section 6) shows both CDF computation and LUT construction completing in under 0.001 ms even on 24-megapixel images — they report as 0.000 ms at the millisecond precision used for logging, in every one of the ten test runs (five sequential, five parallel) collected for this report. This confirms the a-priori reasoning: with total work this small, there is no possible thread-count at which parallelizing these two stages would pay for its own overhead. This is the project's concrete illustration of Amdahl's Law — not as an abstract formula, but as a stage that was excluded from parallelization specifically because the serial fraction of the total workload it represents is negligible in absolute terms, while the fraction of the pipeline's algorithmic structure it represents (a genuine sequential dependency) is real and unavoidable. Section 7 revisits this quantitatively against the measured speedup ceiling.

1. Image I/O: OpenCV

OpenCV was used exclusively for image decoding/encoding (`cv::imread`, `cv::imwrite`) and as a convenient buffer container (`cv::Mat`). All algorithmic logic — grayscale conversion, histogram computation, CDF, LUT construction, and LUT application — was implemented manually rather than via OpenCV's built-in equivalents (e.g. `cv::cvtColor`, `cv::equalizeHist`). This was deliberate: using OpenCV's built-in equalization

would have made the baseline uninstrumentable and non-parallelizable by design, which would have defeated the purpose of the assignment. OpenCV's role was strictly limited to file I/O, which is explicitly outside the parallelization scope described in Section 3.4 and is measured separately in Section 6 (as "loading" and "saving") specifically so that it does not get conflated with the compute stages this project is actually about.

1. Timing Methodology

Timing was implemented using `std::chrono::high_resolution_clock`, instrumented directly around each pipeline stage rather than relying on an external profiler. This was chosen for reproducibility (no additional tooling dependency required) and because it allowed sub-stage instrumentation specific to this pipeline's structure — in particular, splitting the "histogram equalization" stage into its four constituent timings (histogram computation, CDF, LUT build, LUT application) rather than reporting one combined figure, which is what surfaced the negligible cost of the sequential sub-stages reported in Section 3.4.

1. Implementation Details

1. Algorithms

- **Grayscale conversion:** luminosity-weighted conversion, $\text{gray} = 0.299R + 0.587G + 0.114B$, applied per pixel with round-to-nearest and clamping to $[0, 255]$.
- **Histogram computation:** a 256-bin count of grayscale intensity values across the image.
- **CDF computation:** a running (cumulative) sum over the 256-bin histogram.
- **LUT construction:** the standard histogram-equalization normalization, $\text{lut}[i] = \text{round}((\text{cdf}[i] - \text{cdf_min}) / (\text{totalPixels} - \text{cdf_min}) \times 255)$, with a degenerate-image guard (identity LUT) when the denominator is ≤ 0 .
- **LUT application:** a per-pixel remap of the grayscale image through the constructed LUT to produce the final equalized image.

1. Data Structures

- `std::array<long, 256>` for histogram and CDF bins — fixed-size, stack-allocated, cache-friendly, and large enough (long) to avoid overflow on a 24-megapixel single bin count.
- `std::array<uchar, 256>` for the LUT, matching the 8-bit intensity domain it maps.
- `cv::Mat` for image buffers, used purely as a typed, row-addressable pixel buffer (via `.ptr<T>(row)`) rather than through any OpenCV algorithmic API.

1. Parallelization Strategy

All three parallel stages (grayscale conversion, histogram computation, LUT application) use the same row-based static partitioning strategy: the image height is divided into `numThreads` contiguous row ranges (the last thread absorbs any remainder rows), one `pthread` is spawned per range, and the parent thread joins all workers before proceeding. This was chosen over column-based or block-based partitioning because `cv::Mat` stores pixel data in row-major order — row-contiguous partitioning gives each thread a contiguous memory region to scan, which is more cache- and prefetcher-friendly than a partitioning scheme that would force each thread to stride across non-contiguous memory.

Thread count defaults to `std::thread::hardware_concurrency()` (falling back to 4 if the platform cannot report it) and can be overridden via a command-line argument, which is what allowed the benchmarking in Section 6 to be run consistently across test images without recompiling.

1. Synchronization Strategy

As described in Section 3.3, the histogram stage is the only stage with shared mutable state. Each worker thread accumulates into a stack-local `std::array<long, 256>`, and a single `pthread_mutex_t` (`PTHREAD_MUTEX_INITIALIZER`) guards only the final 256-element merge into the shared result array — not the pixel scan itself. Grayscale conversion and LUT application write only to disjoint, per-thread row ranges of the output buffer and therefore require no synchronization at all.

1. Experimental Setup

Parameter	Value
Hardware	Apple Silicon Mac — [M5 Air] — <code>std::thread::hardware_concurrency()</code> reported 10 logical cores
Operating System	macOS [insert version]
Compiler	GCC (installed via Homebrew) — [insert <code>gcc --version</code> output]; Apple Clang was not used because it lacks native OpenMP support and was standardized on for consistency across the pthreads and (earlier, comparative) OpenMP builds
Build system	CMake $\geq$ 3.10, Release build (-O2)
Threading library	POSIX threads (pthreads), linked via CMake's <code>Threads::Threads</code>
Image I/O	OpenCV (<code>cv::imread</code> / <code>cv::imwrite</code>) for decode/encode only
Dataset	5 real (non-synthetic) test images, resolutions ranging from 8.3 MP (3840×2160) to 24.0 MP (4000×6000 / 6000×4000), formats PNG and JPEG
Thread counts tested	10 threads (= <code>hardware_concurrency()</code> on the test machine) for all reported parallel runs
Timing method	<code>std::chrono::high_resolution_clock</code> , wall-clock, per-stage instrumentation, single run per image (see Section 8, Limitations)

1. Results

All timings below are wall-clock milliseconds from a single run per image per implementation, using `std::chrono::high_resolution_clock`. "Compute" refers to grayscale conversion + histogram equalization combined (i.e. excluding image load/save, which are I/O-bound OpenCV calls outside the scope of this project's parallelization — see Section 3.6). Raw per-stage numbers are reported in full below the summary table, per the requirement to report raw numbers rather than only derived charts.

1. Summary: Compute-Only Time and Speedup

Image	Resolution (px)	Megapixels	Seq. Compute (ms)	Par. Compute (ms)	Compute Speedup
Img 1	4000 × 6000	24.0	51.199	11.462	4.47×
Img 2	6000 × 4000	24.0	28.775	8.545	3.37×
Img 3	6015 × 3382	20.3	31.134	8.384	3.71×
Img 4	3840 × 2160	8.3	10.243	3.469	2.95×
Img 5	3840 × 2160	8.3	11.047	3.522	3.14×

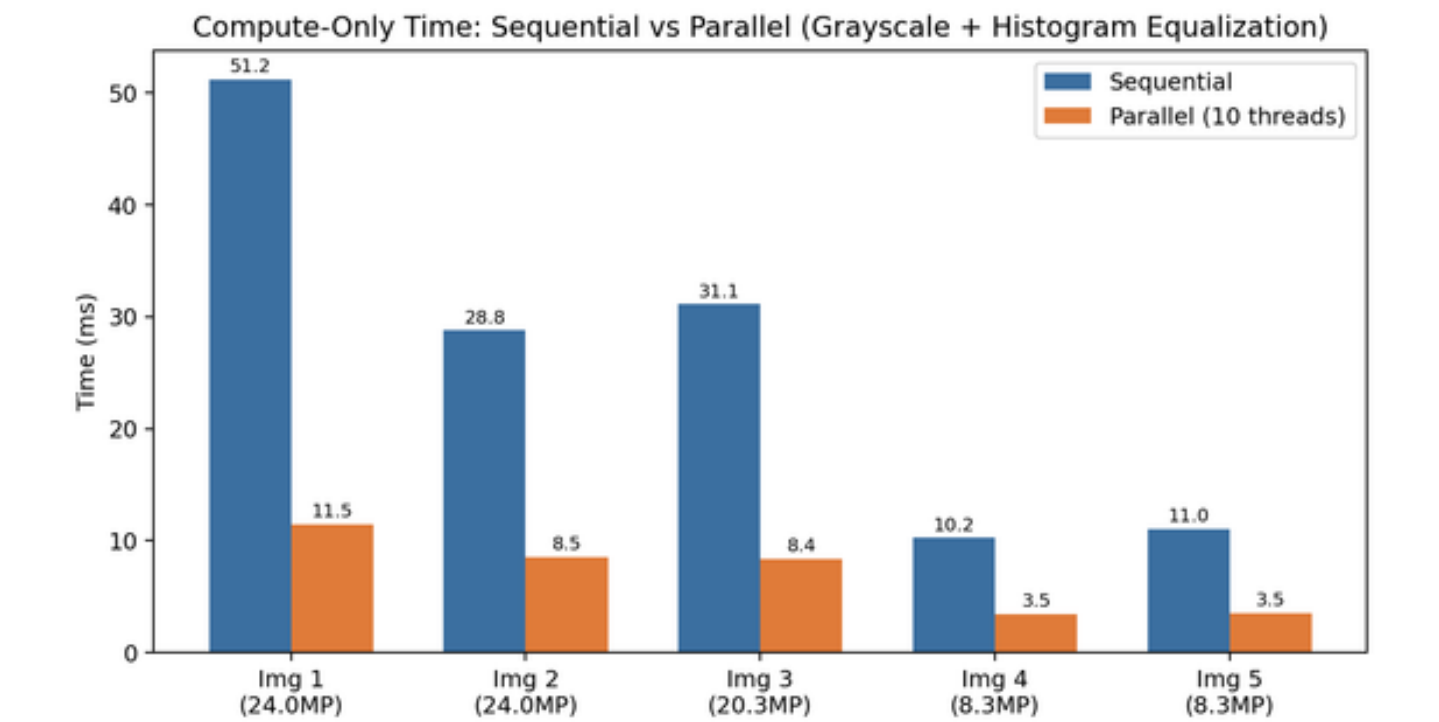


Figure 1. Sequential vs. parallel compute-only time (grayscale + histogram equalization) per test image.

1. Raw Per-Stage Timings — Sequential

Image	Loading (ms)	Grayscale (ms)	Hist. Equal. (ms)	Saving (ms)	Total (ms)	Total. Excl. I/O(ms)
Img 1	77.863	14.693	36.506	135.282	264.345	51.199
Img 2	42.897	14.224	14.551	194.655	266.326	28.775
Img 3	76.877	12.325	18.809	146.829	254.840	31.134
Img 4	61.347	4.771	5.472	66.553	138.143	10.243
Img 5	60.188	4.774	6.273	64.790	136.026	11.047

1. Raw Per-Stage Timings — Parallel (10 threads)

Image	Loading (ms)	Grayscale (ms)	Hist. Equal. (ms)	Saving (ms)	Total (ms)	Total. Excl. I/O(ms)
Img 1	79.386	4.683	6.779	141.935	232.783	11.462
Img 2	42.823	4.306	4.239	203.486	254.854	8.545
Img 3	73.719	3.734	4.651	159.132	241.236	8.385
Img 4	61.755	1.819	1.649	69.696	134.919	3.468
Img 5	60.188	1.623	1.899	68.310	132.020	3.522

1. Stage 3 Breakdown (Parallel Run): Histogram Computation vs. LUT Application

The histogram-equalization stage was further instrumented into its four constituent sub-steps. CDF computation and LUT construction measured 0.000 ms (below the clock's reporting precision) in every single run, sequential and parallel alike — the empirical basis for the Amdahl's Law design decision in Section 3.4.

Image	Histogram Comp. — parallel (ms)	CDF — sequential (ms)	LUT build — sequential (ms)	LUT apply — parallel (ms)
Img 1	4.185	0.000	0.000	2.594
Img 2	1.707	0.000	0.000	2.531
Img 3	2.443	0.000	0.000	2.208
Img 4	0.753	0.000	0.000	0.896
Img 5	0.999	0.000	0.000	0.901

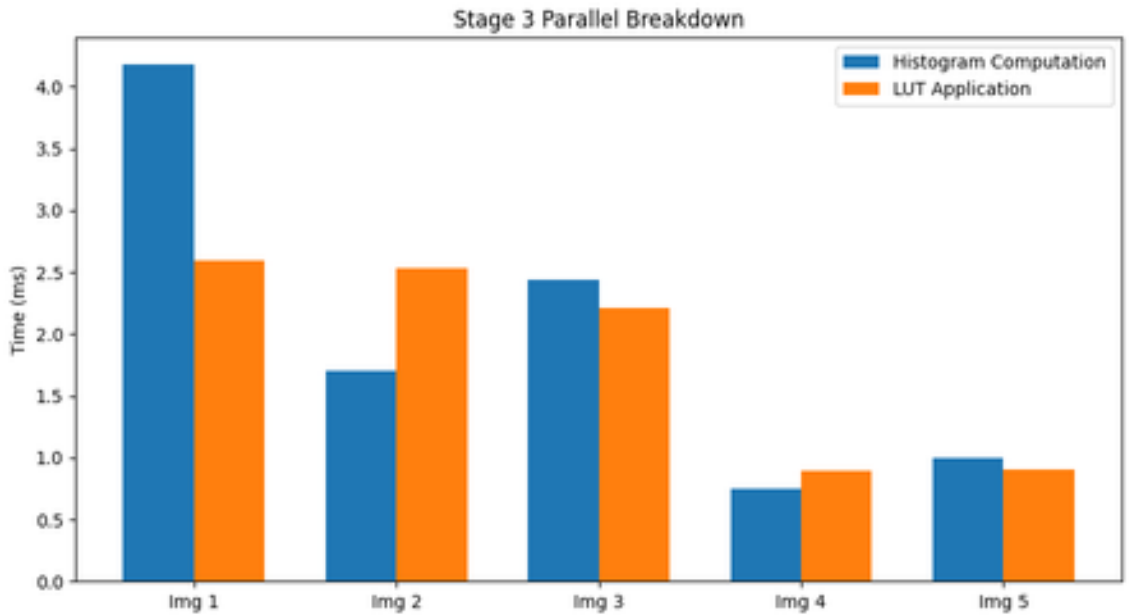
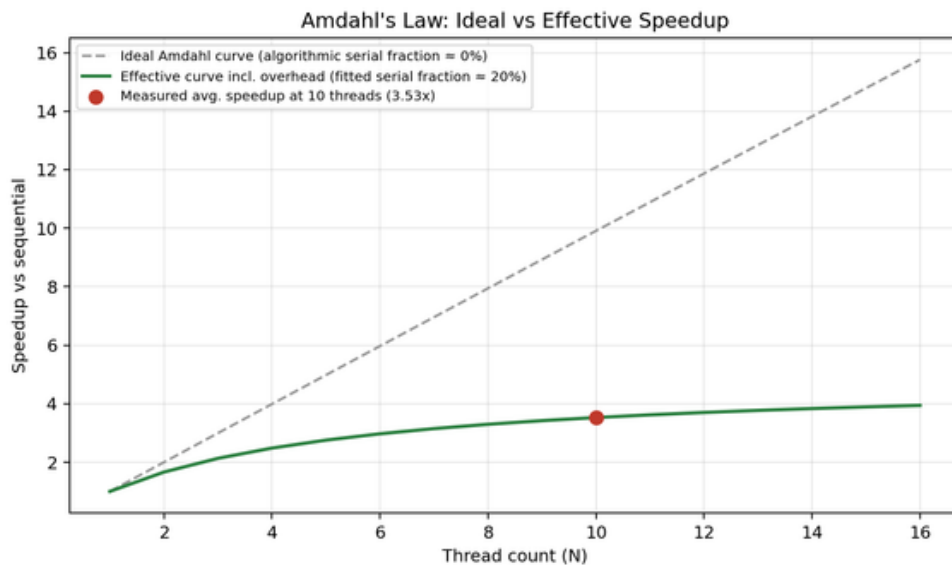


Figure 2. Parallel stage-3 breakdown: histogram computation vs. LUT application, per image

1. Speedup Curve and Amdahl's Law Framing

Only one thread count (10, the machine's reported hardware concurrency) was benchmarked across all five images, so the figure below is not a classic "speedup vs. thread count" sweep on this dataset. Instead it plots two theoretical Amdahl's Law curves against thread count, using serial fractions derived from this project's own measurements, with the actual measured result overlaid as a single point: an ideal curve using the algorithmic serial fraction implied by Section 3.4 ($\approx 0\%$, since CDF+LUT time is negligible), and an effective curve whose serial fraction ($\approx 20\%$) is fitted to reproduce the actual average measured speedup at 10 threads ($3.53\times$, the mean of the six compute-speedup values in Section 6.1). The gap between the two curves is the subject of Section 7's discussion.



1. Discussion

1. Why speedup falls well short of 10×

With 10 threads and a near-zero algorithmic serial fraction (Section 3.4), a naïve reading of Amdahl's Law predicts speedup approaching 10×. The measured compute-only speedup instead ranges from 2.95× to 4.47× across the five test images (mean 3.53×, Section 6.1) — this is the single most important number in this report to explain, not just state, and it is explained by costs that Amdahl's simple two-term model does not account for at all:

- **Thread creation/join overhead.** Each parallel stage spawns and joins 10 pthreads from scratch — there is no persistent thread pool. Across grayscale, histogram, and LUT-apply, that is up to 30 pthread_create/pthread_join round-trips per image. For the smaller test images (Img 4, Img 5 at 8.3 MP), the entire compute stage completes in 10–11 ms sequentially; the fixed cost of thread lifecycle management is a proportionally larger fraction of a shorter job, which matches the lower speedup observed on the smaller images (2.95×–3.14×) relative to the larger ones (3.37×–4.47×).
- **Memory bandwidth ceiling.** Grayscale conversion and LUT application are memory-bound, not compute-bound — each pixel requires a handful of arithmetic operations but a full read and write. All 10 threads share the same memory bus and last-level cache; beyond a small number of threads, additional threads contend for bandwidth rather than adding useful throughput, which caps realistic scaling well below the core count regardless of how cleanly the work is partitioned.
- **Heterogeneous core architecture.** Apple Silicon chips expose a mix of performance and efficiency cores under a single hardware_concurrency() count. Static row-based partitioning assigns each of the 10 threads an equal number of rows, but if some of those 10 logical threads are scheduled onto slower efficiency cores, the workload is not actually balanced 10 ways evenly in time — the stage's completion time is bound by its slowest thread. This is a plausible contributor to the sub-linear scaling and is a concrete candidate for the profiling work listed in Section 9 (Future Work).
- **The histogram merge is not entirely free.** The single mutex-protected merge (Section 3.3) is a fixed 256-addition critical section per thread — small, but not zero, and it is on the pipeline's critical path once every thread has finished its local scan.

Framed against Amdahl's Law directly: fitting the two-term model to the measured 3.53× average at 10 threads implies an “effective” serial fraction of roughly 20% (Figure 3) — far higher than the true algorithmic serial fraction of the CDF/LUT stages ($\approx 0\%$, Section 3.4). That 20-point gap is not algorithmic serial work at all; it is the aggregate cost of the overheads above, behaving like serial time under Amdahl's model even though none of it is a genuine sequential dependency in the algorithm. This is, in our view, the report's central design-vs-reality talking point: the design correctly minimized true algorithmic serial fraction to near zero, and the remaining gap to ideal speedup is entirely an engineering-overhead problem, not an algorithm-design problem.

1. Why end-to-end (I/O-inclusive) speedup is much smaller than compute speedup

An obvious follow-up question would be: if compute speedup is up to $4.47\times$, why is total wall-clock speedup (sequential total $\div$ parallel total, Section 6.2 vs. 6.3) only about $1.02\times$ – $1.14\times$ across the same five images? The answer is structural, not a flaw: image loading and saving are OpenCV-delegated I/O calls that were deliberately left outside the parallelization scope (Section 3.6), and they dominate total wall-clock time — for example, Img 2's save step alone (194–203 ms) is roughly 7–10 $\times$ longer than its entire compute stage (4–9 ms) in both implementations. Parallelizing a stage that is 3–4% of total wall-clock time, however successfully, cannot move total wall-clock time by more than that same 3–4%. This is an expected behaviour given the project's explicitly compute-scoped design, not an unexplained anomaly, but it is worth stating plainly so the compute-stage speedup numbers in Section 6.1 are not misread as end-to-end throughput claims.

1. Anomaly: Image 1's sequential histogram-equalization time

Image 1 and Image 2 are both 24.0-megapixel images, so their per-pixel work is identical in principle. Grayscale conversion times are consistent with this (14.693 ms vs. 14.224 ms, a 3% difference). Sequential histogram-equalization time is not: 36.506 ms for Image 1 versus 14.551 ms for Image 2 — roughly 2.5 $\times$ higher for the same amount of arithmetic work. The parallel run shows the same directional effect at a smaller scale (6.779 ms vs. 4.239 ms). Because the equalization algorithm's cost is a fixed function of pixel count and does not depend on pixel values, this points to a measurement artifact rather than an algorithmic one: Image 1 was the first test run executed after building the sequential binary, and cold caches (both OS file-system cache and CPU cache/TLB state, not yet warmed by a prior run) plus CPU frequency ramp-up on Apple Silicon's dynamic clocking are a well-documented source of inflated first-run timings. We flag this as an artifact of the benchmarking methodology (single run per image, no warm-up run discarded) rather than a property of the algorithm or the implementation, and treat it as a limitation of the current measurement process (Section 8) rather than omit it from the results.

1. Limitations

- Single run per image/implementation. All timings in Section 6 are from one run each, not an average over multiple runs. As shown in Section 7.3, this leaves at least one measurement (Image 1's sequential histogram-equalization time) vulnerable to a cold-start artifact. Repeated runs with the first (warm-up) run discarded would produce tighter, more defensible numbers.
- Single thread-count tested. All parallel results use 10 threads. No sweep across thread counts (e.g. 2, 4, 8, 16) was benchmarked, so the “speedup curve” in Section 6.5 is a fitted theoretical curve anchored to one measured point, not an empirically measured curve across thread counts.
- No multi-core scaling test beyond one machine. All results are from a single Apple Silicon development machine. No cross-platform (Linux/x86) or cross-core-count comparison was performed, so it is not established how these results generalize to different hardware.
- I/O is out of scope by design (Section 3.5), which means this project does not address — and makes no claims about — optimizing image decode/encode time, which dominates end-to-end latency (Section 7.2).
- No adversarial or malformed-input testing. All five test images are well-formed, real photographs; behaviour on corrupted files, unsupported formats, or extreme aspect ratios has not been tested beyond OpenCV's own error handling.
- Not tested at multi-node or distributed scale, and no security hardening (e.g. against malicious image files) was in scope for this assignment.

2. Future Work

- Re-run the benchmark suite with multiple iterations per image (discarding a warm-up run) and report mean $\pm$ standard deviation, to resolve the cold-start anomaly identified in Section 7.3 and tighten the speedup figures in Section 6.
- Sweep thread count (1, 2, 4, 8, 10, 16) to produce a genuinely measured speedup-vs-threads curve, replacing the single-point-fitted curve in Figure 3 with real data, and to test the heterogeneous-core hypothesis raised in Section 7.1 directly (e.g. by pinning threads to performance vs. efficiency cores).
- Re-implement and separately benchmark the atomic-based histogram variant as its own binary, to convert the qualitative claim in Section 3.3 into a quantified, reportable result.

- Add a parallel prefix-sum (e.g. Hillis-Steele) implementation of the CDF stage purely as a comparative exercise, even though Section 3.4 shows it would not be worthwhile at this problem size — useful to quantify exactly how much thread-spawn overhead would need to shrink before it would be.
- Camera RAW format support via LibRaw, extending the current PPM/PNG/JPEG/BMP input support — identified as an optional stretch goal, not a core deliverable for this submission.
- Cross-platform benchmarking on a Linux/x86 machine to test whether the speedup ceiling observed here is Apple Silicon-specific (heterogeneous cores, unified memory bandwidth) or a general property of this workload and thread-count.

3. Individual Contributions

Team Member	Primary Contributions
[Shivalal Shibu]	Sequential as well as Parallel Implementation, with architecture design
[Basil Varghese]	Parallel implementation (pthreads), synchronization strategy design and mutex/atomic comparison
[Shravan Raveendran Nair Santhini]	Benchmarking harness, timing instrumentation, chart generation, report authorship

1. References

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. AFIPS Conference Proceedings.

Gonzalez, R. C., & Woods, R. E. Digital Image Processing. Chapter on histogram processing and equalization.

OpenCV documentation — cv::Mat, cv::imread, cv::imwrite. <https://docs.opencv.org/>

POSIX Threads Programming, Lawrence Livermore National Laboratory tutorial. <https://hpc-tutorials.llnl.gov/posix/>

CMake documentation — find_package, Threads module. <https://cmake.org/documentation/>